REPORT 02 / 18 OCTOBER 2026

Technical Approach Report

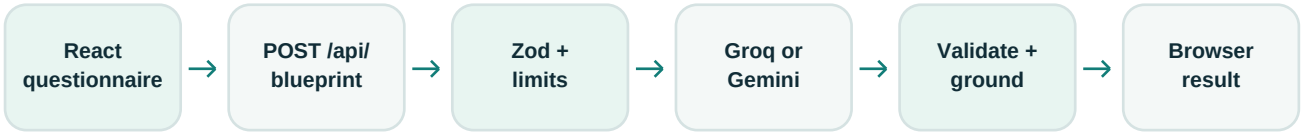
Implementation review of the Cynbit AI Solutions Architect proof of concept, with a reproducible evaluation plan and explicit limits.

Prepared by Suhail Kataria | 18 October 2026

System purpose

A Next.js 16 / React 19 web application guides a user through business discovery and requests one structured solution blueprint from a configured provider. The repository implements Groq with **openai/gpt-oss-120b** and an optional Gemini adapter with **gemini-2.5-flash-lite**. The provider is a server environment choice; the browser cannot choose the model [1].

Actual request path



The diagram describes the implemented PoC. There is no database, authentication, retrieval layer, agent tool chain, or customer-system connector. The model receives the normalized questionnaire in a single prompt; it does not independently verify client claims or fetch external facts [1].

Core implementation map

Layer	Repository evidence	Responsibility
UI	src/components/workspace and discovery	Stages, review, result display, copy and JSON export.
API	src/app/api/blueprint/route.ts; handler.ts	Request bounds, validation, limits, cache and safe errors.
AI boundary	providers.ts; schema.ts; grounding.ts	Locked model adapters, JSON schema, output and KPI checks.
Persistence	storage.ts; generate.ts	Browser-local draft/result and attempt ledger.

Evidence status: source and test definitions were inspected. One separately documented live run appears on page 5; it does not establish typical latency, reliability, output quality or business impact.

02 / DISCOVERY DATA

Input and data pipeline

Questionnaire

The three stages are Business, Challenge and Requirements, followed by a review screen. Eight primary items are required: industry, description, problem, workflow, challenges, intended users, goals and success criteria. Optional fields include tools, size, data availability, integration, security constraints, timeline and budget. Focus selection conditionally reveals support volume, recruitment applications, manual-work hours or SaaS product status [1].

Deterministic preprocessing

A shared Zod schema validates drafts and complete submissions. Conditional values hidden by a changed focus are cleared during normalization. 'Other' choices require detail; 'No data yet' and 'Unsure' are exclusive selections. The budget amount is optional; a separate currency selection supports INR, USD, EUR, GBP, AED, CAD, AUD, SGD and JPY. An empty amount remains unspecified rather than implying an INR budget [1].

Generation sequence

Step	Enforced behavior
1. Review	User sees normalized answers; navigation and page load make no AI request.
2. Submit	Client validates, checks the 32 KiB body cap, stores draft and counts one attempted POST.
3. Server gate	Route checks origin, JSON, UUID/session, questionnaire schema, provider setup and in-memory limits.
4. Model call	One configured provider request; no automatic retry, continuation or fallback.
5. Accept	Parse JSON, enforce output schema and size, ground KPI provenance, attach metadata, cache response.
6. Revisit	Successful result is cached in browser; copy/export/revisit do not regenerate.

A browser Web Lock and an immediate click guard prevent concurrent tabs from submitting through the normal UI. The server also rejects the same session while pending. Neither mechanism authenticates a caller [1].

03 / MODEL BOUNDARY

Prompting and structured output

Model and provider tradeoff

The configured Groq adapter uses the official SDK, model openai/gpt-oss-120b, low reasoning effort and strict JSON Schema output. Groq documents structured-output support for this model [2–3]. This is a practical selection for one-shot, structured synthesis; the repository does not contain a comparative quality or cost benchmark. A Gemini adapter exists for explicit server configuration, not automatic fallback [1]. Provider latency, cost and availability remain external dependencies.

Prompt contract

The system instruction treats answers as untrusted data, asks for grounded recommendations, allows a non-AI recommendation, forbids invented ROI, cost, baselines and achieved outcomes, and requires assumptions and missing information. The user message is a serialized questionnaire. No tool calls, browser research or retrieval occur during generation. This reduces some failure modes but cannot guarantee factual or commercial correctness [1].

Blueprint schema, summarized

Group	Required content
Decision	Executive summary; problem/requirements; solution and justification; AI feasibility.
Design	Stack components with rationale; architecture overview and data-flow steps.
Plan	Roadmap phases with activities, deliverables and exit criteria; next steps.
Caveats	Separate risks, assumptions, missing information and limitations.
Value	Benefits; KPIs with metric, measurement, nullable baseline, target and target source.

Zod emits a provider-facing JSON schema with common keywords, then validates the returned object again with stricter local bounds. The response is limited to 64 KiB and 12,000 output tokens; a finish reason other than normal completion fails without repair [1].

04 / GROUNDING

KPI integrity and validation

Known versus proposed numbers

A generated KPI baseline is kept only when its exact normalized text appears within a supplied questionnaire value; otherwise it becomes null. A claimed user-provided target is relabeled 'needs clarification' unless the target text appears in the answers. The original success-criteria field is copied into a separate user-provided row so an AI rewrite cannot erase it. AI-proposed targets remain suggestions and are never displayed as measured outcomes [1].

Example of expected labeling

Input / output	Displayed interpretation
User writes: 'Reduce weekly manual work from 20 hours to 8 hours.'	Original goal is preserved as user-provided; the 20-hour baseline must be explicitly supported.
Model suggests: 'Reach 90% first-response within 2 hours.'	AI recommendation, not an achieved result or user commitment.
Model invents: 'Current rate is 65%.'	Baseline becomes 'Not provided' unless the exact value appears in the intake.

Limits of this check

The substring check is deliberately conservative: it may reject a valid paraphrase and cannot confirm that user-supplied numbers are true. It also cannot prove that a model's rationale logically follows. A reviewer must inspect sources, units, time windows and feasibility before accepting any target. Legacy saved results are read without rewriting or regenerating them; older provenance fields may be absent [1].

Output and failure path

Server-side validation rejects incomplete or oversized JSON and emits a generic public error. Answers remain saved for an explicit retry. The browser only shows the route's short error contract, not arbitrary upstream HTML. Generation metadata records provider, model, timestamp, status, duration and provider-reported token counts where available; missing counts remain unavailable rather than estimated [1].

05 / OBSERVED EXECUTION

Live Model Execution: Verified Test Result

A candidate-documented request for the retail/e-commerce scenario returned a blueprint that the application accepted and rendered. The server acceptance path validates generated JSON with Zod before returning it [1]. The recorded timestamp is **18 October 2026, 2:18:57 PM**; the supplied run metadata is shown below [4]. The screenshots show the rendered blueprint and architecture, while the metric values come from the supplied run record, not from the screenshots [4–5].

Metric	Observed Value
Provider	Groq
Model	openai/gpt-oss-120b
Input tokens	1,601
Output tokens	2,612
Total tokens	4,213
Generation time	6.018 seconds
Status	Success
Test date	October 18, 2026

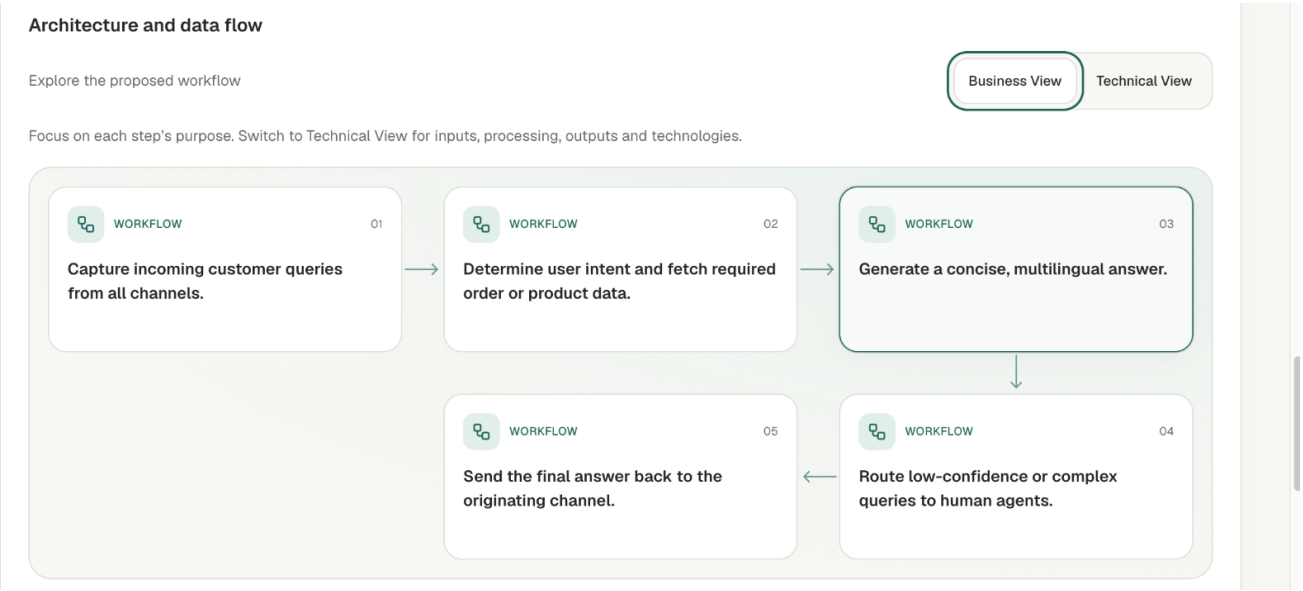


Figure 1. Business View of the saved, five-step proposed workflow. The screenshot shows a rendered design proposal, not a deployed support system [5].

The generated stages were **Message Ingestion, Intent & Data Retrieval, Response Generation, Escalation Decision** and **Delivery**. The user reviewed one intake, triggered one provider request, then viewed the saved result. Business View and Technical View are frontend presentations of that same saved architecture; switching views makes no additional model call [1, 5].

These figures describe one successful request only. They do not establish average latency, reliability, cost or output quality across scenarios. The application generated a proposed customer-support solution blueprint; it did not implement the proposed chatbot.

06 / SAFETY AND OPERATIONS

Controls and residual risks

Area	Implemented PoC control	Residual risk / production need
Secrets	Provider key read only on server; client cannot select provider/model.	Secret management and rotation required for hosting.
Request abuse	32 KiB body; 5 s body read; 55 s generation; per-process daily/minute/concurrency limits.	No auth; process restarts and multiple instances reset limits. Add shared store and identity.
Idempotency	Session UUID, pending guard, 24 h server cache up to 128 results; browser saved result.	No durable exactly-once billing guarantee across instances or eviction.
Data privacy	No questionnaire/raw provider logging in application code; no-store response; browser localStorage.	LocalStorage is unencrypted; provider receives answers. Define consent, retention and DPA before real data.
Presentation	Structured React text rendering, no raw model HTML in results.	Human review still needed for inaccurate or unsafe recommendations.

Threat model in scope

The main risks are prompt injection inside questionnaire text, costly repeated requests, sensitive business data in a browser profile or external provider, and overconfident output. The prompt explicitly marks answers as data; schema and server checks constrain format and size; rate controls reduce casual abuse. These controls do not make the endpoint suitable for public unrestricted access or confidential client submissions [1].

Operational decision

For a private demonstration, use synthetic or non-sensitive scenarios and server-only credentials. Before public deployment, add authentication, durable shared quotas and session records, telemetry without sensitive content, retention controls, provider data-processing review and an architect approval step. These are proposed work, not current features.

07 / REPRODUCIBILITY

Testing, evaluation and next steps

Reproducible local setup

Use Node.js 22.13 or newer. In the project directory run `npm ci`, copy `.env.example` to a local environment file, set `AI_PROVIDER=groq` and `GROQ_API_KEY` server-side, then run `npm run dev` and open `/workspace`. The repository also documents `npm run check`, `npm run test:unit` and `npm run test:e2e`. Never place the key in a `NEXT_PUBLIC_` setting [1].

What has been verified for this report

The source tree contains unit tests for required fields, hidden conditional values, currency budgets, session caching, one-request behavior, schema/KPI provenance, usage metadata and mocked SDK error paths. Playwright specs cover discovery, theme and homepage behavior. These tests were inspected, not rerun for this report. They mock provider calls, so their results cannot establish live model quality or business benefit. One separately documented successful live request appears on page 5 [1, 4–5].

Proposed evaluation protocol

Track	Method and reporting
Functional	Run lint/typecheck/build, unit suite and desktop/mobile browser flows on a fixed commit; record exact counts and failures.
Quality	20–30 consented or synthetic briefs across industries; two reviewers score groundedness, omissions, usefulness and non-AI appropriateness. Report disagreements.
Robustness	Probe empty/ambiguous inputs, prompt injection, unknown baselines, mixed currencies, quota errors and malformed responses.
Operations	Record p50/p95 latency, success rate, reported tokens and actual provider charges from a dated live trial; compare to a manual template.

Roadmap: run baseline evaluation; review failures; add authenticated shared state and privacy controls; then trial a human-approved discovery workflow. The single documented live request is not a deployment or a performance benchmark.

References

[1] Local Cynbit repository: README, package.json, src/lib/discovery, src/lib/blueprint, src/components/workspace and tests, inspected 18 Oct 2026.
[2] Groq, Structured Outputs, accessed 18 Oct 2026.
[3] Groq, GPT OSS 120B model page, accessed 18 Oct 2026.
[4] Candidate-supplied metadata for one successful Groq request, 18 Oct 2026, 2:18:57 PM. Provider logs were not independently audited.
[5] Candidate-supplied application architecture screenshot, 18 Oct 2026. Cropped for legibility; displayed content remains unaltered.